

Tuyển Tập Bài Tập: Kỹ Thuật Slope Trick trong Quy Hoạch Động

(16 Bài toán Tuyển chọn Phân cấp từ Codeforces, AtCoder, CSES, BalticOI & APIO)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 **Duc-Minh Vu**. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Bảng Tổng Hợp 16 Bài Tập Tuyển Chọn

STT	Mã Bài / Tên Bài	Nguồn / Nền tảng	Cấp độ	Dạng bài & Kỹ thuật cốt lõi	Độ phức tạp
1	CSES 1074 (Stick Lengths)	CSES Problemset	Khởi động	Tổng khoảng cách $\sum x - p_i $ & Điểm trung vị	$O(N \log N)$
2	AtCoder ABC 127 F (Absolute Minima)	AtCoder Beginner Contest 127	Khởi động	Thêm hàm $ x - a + b$, truy vấn $\arg \min$ qua 2 Heaps (L, R)	$O(Q \log Q)$
3	Codeforces 867E (Buy Low Sell High)	Codeforces Round 417	Khởi động	Tối ưu danh mục cổ phiếu $\max(0, x - p)$, bù trừ hệ số góc	$O(N \log N)$
4	Codeforces 713C (Sonya and Problem Wihtout a Legend)	Codeforces Round 371	Khởi động	Đổi biến dãy tăng ngặt $A'[i] = A[i] - i$, duy trì Max-Heap L	$O(N \log N)$
5	Codeforces 13C (Sequence)	Codeforces Beta Round 13	Trung cấp	Dãy không giảm số nguyên, tiền thân của Slope Trick hiện đại	$O(N \log N)$
6	AtCoder ARC 070 E (NarrowRectangles)	AtCoder Regular Contest 070	Trung cấp	Cơ bản cửa sổ đoạn thẳng, Lazy Tags <code>shift_L</code> & <code>shift_R</code>	$O(N \log N)$
7	AtCoder ABC 217 H (Snaketoy)	AtCoder Beginner Contest 217	Trung cấp	Di chuyển né đòn $\leq \Delta t$, thêm hàm dốc trái/phải đối xứng	$O(N \log N)$

STT	Mã Bài / Tên Bài	Nguồn / Nền tảng	Cấp độ	Dạng bài & Kỹ thuật cốt lõi	Độ phức tạp
8	Luogu P4331 (【BalticOI 2004】Sequence)	BalticOI 2004 / Luogu	Trung cấp	Biến đổi dãy tăng nghiêm ngặt & truy vết không phục dãy B tối ưu	$O(N \log N)$
9	AtCoder ARC 123 D (Inc, Dec - Decomposition)	AtCoder Regular Contest 123	Nâng cao	Tách $A_i = B_i + C_i$ ($B \uparrow, C \downarrow$) min $\sum(B_i + C_i)$ qua Slope Trick	$O(N \log N)$
10	Codeforces 1251E2 (Voting - Hard Version)	Educational Round 75	Nâng cao	Mua phiếu bầu tối thiểu, phân tích tính lồi của đường chi phí	$O(N \log N)$
11	Codeforces 802N (April Fools' Problem - Medium)	Helvetic Coding Contest 2017	Nâng cao	Ghép cặp k công việc (chuẩn bị & in), Greedy kết hợp hàm chi phí lồi	$O(N \log N)$
12	AtCoder AGC 018 C (Coins)	AtCoder Grand Contest 018	Nâng cao	Phân phối 3 loại đồng tiền (X, Y, Z) , tối ưu hóa 2 đầu bằng Heap	$O(N \log N)$
13	Luogu P3642 / APIO 2016 (Fireworks - 烟花表演)	APIO 2016 / Luogu	Chinh phục	Slope Trick trên Cây: Gộp hàm lồi bằng Heap có thể trộn (Leftist Heap)	$O(N \log N)$
14	Codeforces 802O (April Fools' Problem - Hard)	Helvetic Coding Contest 2017	Chinh phục	Phối hợp Alien's Trick (WQS Binary Search) và Slope Trick	$O(N \log N \log C)$
15	Luogu P5324 (【BJOI2019】删除 - Delete)	BJOI 2019 / Luogu	Chinh phục	Sửa đổi mảng thành hoán vị, biểu diễn hình học độ dốc trên Segment Tree	$O((N + M) \log N)$
16	Codeforces 1148E (Earth Wind and Fire)	Codeforces Round 566	Chinh phục	Cân bằng tọa độ không đổi thứ tự, bao lồi tích lũy & Ghép cặp Stack/Heap	$O(N \log N)$

1. Cấp Độ 1: Khởi Động & Nền Tảng (Warm-up & Fundamentals)

Bài 1: [CSES 1074 - Stick Lengths](#)

- **Nguồn:** [CSES Problemset — Sorting and Searching](#).
- **Giới hạn:** $N \leq 2 \times 10^5, p_i \leq 10^9$, Thời gian: 1.0s, Bộ nhớ: 512 MB.
- **Tóm tắt bài toán:** Cho N thanh gỗ có độ dài $p_1, p_2, \dots, p_N$. Tìm độ dài chung x sao cho tổng chi phí biến đổi $\sum_{i=1}^N |x - p_i|$ là nhỏ nhất.

Gợi ý 3 bậc:

- **Bậc 1 (Góc nhìn hình học & Hàm lỗi):** Hàm mục tiêu $f(x) = \sum_{i=1}^N |x - p_i|$ là tổng của N hàm trị tuyệt đối lỗi. Do đó $f(x)$ là một hàm lỗi liên tục từng khúc.
- **Bậc 2 (Điểm đổi độ dốc):** Tại $x < \min(p)$, hệ số góc của $f(x)$ là $-N$. Mỗi khi x vượt qua một giá trị p_i , hệ số góc tăng thêm $+2$.
- **Bậc 3 (Điểm cực tiểu):** Hệ số góc đổi dấu từ âm sang không âm tại điểm chính giữa (phần tử trung vị - Median). Sắp xếp mảng p và chọn $x = p[N/2]$. Độ phức tạp $O(N \log N)$ hoặc $O(N)$ dùng `std::nth_element`.

Bài 2: [AtCoder ABC 127 F - Absolute Minima](#)

- **Nguồn:** [AtCoder Beginner Contest 127 — Task F](#).
- **Giới hạn:** $Q \leq 2 \times 10^5$, $a_i, b_i \in [-10^9, 10^9]$, Thời gian: 2.0s, Bộ nhớ: 1024 MB.
- **Tóm tắt bài toán:** Bắt đầu với hàm $f(x) = 0$. Có 2 loại truy vấn:
 - Thêm vào hàm số: $f(x) \leftarrow f(x) + |x - a| + b$.
 - Tìm giá trị x nhỏ nhất làm $f(x)$ đạt giá trị nhỏ nhất, đồng thời in ra giá trị nhỏ nhất đó.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện Slope Trick thuần túy):** Hàm $f(x)$ luôn có dạng lồi đối xứng ở hai bên cực tiểu. Độ dốc bên trái giảm dần từ 0 về $-\infty$, bên phải tăng dần từ 0 lên $+\infty$.
- **Bậc 2 (Duy trì 2 Priority Queues):** Sử dụng Max-Heap L (chứa các điểm dốc ≤ 0) và Min-Heap R (chứa các điểm dốc ≥ 0). Cân bằng sao cho kích thước $|L| = |R|$ (nếu số điểm chẵn) hoặc $|L| = |R| + 1$ (nếu số điểm lẻ). Khi đó, $\arg \min$ luôn là đỉnh của L : $x^* = \max(L)$.
- **Bậc 3 (Cập nhật giá trị cực tiểu):** Mỗi khi thêm $|x - a|$, duy trì tổng các giá trị trong L và R , hoặc tính phần gia số $\Delta f_{\min} = \max(0, \max(L) - a)$ hoặc $\max(0, a - \min(R))$. Cộng dồn b vào biến độc lập $f_{\min}$. Độ phức tạp $O(Q \log Q)$.

Bài 3: [Codeforces 867E - Buy Low Sell High](#)

- **Nguồn:** [Codeforces Round 417 \(Div. 2\) — Problem E](#).
- **Giới hạn:** $N \leq 3 \times 10^5$, $p_i \leq 10^6$, Thời gian: 2.0s, Bộ nhớ: 256 MB.
- **Tóm tắt bài toán:** Cho giá cổ phiếu trong N ngày liên tiếp. Mỗi ngày được phép: mua 1 cổ phiếu, bán 1 cổ phiếu đã có, hoặc không làm gì. Tìm lợi nhuận tối đa.

Gợi ý 3 bậc:

- **Bậc 1 (Thiết lập hàm DP):** Gọi $dp[i][j]$ là lợi nhuận lớn nhất sau ngày i khi đang nắm giữ j cổ phiếu. Xét hàm theo j : $f_i(j)$ là hàm lồi theo j .
- **Bậc 2 (Slope Trick với Min-Heap):** Nếu ta coi việc mua cổ phiếu giá p_i là thêm một quyền chọn bán với giá p_i , mỗi ngày khi xuất hiện giá p_i , ta kiểm tra xem có cổ phiếu nào mua trước đó với giá $u < p_i$ không.

- **Bậc 3 (Đẩy đúp để hỗ trợ "Hối hận" - Regret):** Nếu $p_i > \min(pq)$, ta ghép cặp bán cổ phiếu rẻ nhất: lời $p_i - u$. Sau đó đẩy lại p_i vào Heap **hai lần**: một lần biểu thị cổ phiếu này đã được bán (nhưng có thể "hối hận" để bán ở giá cao hơn trong tương lai), một lần là cơ hội mua cổ phiếu giá p_i từ đầu. Độ phức tạp $O(N \log N)$.

Bài 4: [Codeforces 713C - Sonya and Problem Without a Legend](#)

- **Nguồn:** [Codeforces Round 371 \(Div. 1\) — Problem C](#).
- **Giới hạn:** $N \leq 3000$ (có thể mở rộng lên $N \leq 5 \times 10^5$), $a_i \leq 10^9$, Thời gian: 5.0s, Bộ nhớ: 256 MB.
- **Tóm tắt bài toán:** Cho dãy số A . Cần biến đổi thành dãy B tăng nghiêm ngặt ($B_1 < B_2 < \dots < B_N$) sao cho $\sum_{i=1}^N |A_i - B_i|$ nhỏ nhất.

Gợi ý 3 bậc:

- **Bậc 1 (Đổi biến sang không giảm):** Điều kiện $B_i < B_{i+1} \iff B_i - i \leq B_{i+1} - (i + 1)$. Đặt $A'_i = A_i - i$ và $B'_i = B_i - i$. Bài toán quy về biến đổi A' thành dãy B' không giảm.
- **Bậc 2 (Công thức DP và phép co tiền tố):** $dp[i][x] = \min_{y \leq x} dp[i-1][y] + |x - A'_i|$. Phép toán $\min_{y \leq x} f(y)$ tương đương với việc "cắt bỏ" toàn bộ các đoạn có độ dốc dương (> 0). Do đó ta chỉ cần duy trì các điểm chuyển dốc của sườn trái thông qua một Max-Heap L .
- **Bậc 3 (Thao tác Heap):** Khi xét đến A'_i :
 - Nếu $A'_i < \max(L)$, ta chịu thêm chi phí $\max(L) - A'_i$, lấy đỉnh ra khỏi L và đẩy A'_i vào L hai lần (một để sửa điểm gãy, một làm điểm dốc mới).
 - Nếu $A'_i \geq \max(L)$, chỉ cần đẩy A'_i vào L một lần.
 - Tổng thời gian: $O(N \log N)$, bộ nhớ $O(N)$.

2. Cấp Độ 2: Trung Cấp & Dịch Chuyển Hàm Số (Intermediate & Function Shifts)

Bài 5: [Codeforces 13C - Sequence](#)

- **Nguồn:** [Codeforces Beta Round 13 — Problem C](#).
- **Giới hạn:** $N \leq 5000$, $|A_i| \leq 10^9$, Thời gian: 1.0s, Bộ nhớ: 64 MB.
- **Tóm tắt bài toán:** Biến đổi dãy A thành dãy không giảm với tổng chi phí $\sum |A_i - B_i|$ nhỏ nhất.

Gợi ý 3 bậc:

- **Bậc 1 (So sánh với CF 713C):** Bản chất bài này chính là bài 713C nhưng dãy đích là không giảm ($B_1 \leq B_2 \leq \dots \leq B_N$), không cần trừ chỉ số i .
- **Bậc 2 (Mô hình hàm dốc):** Hàm số $f_i(x) = \min_{y \leq x} f_{i-1}(y) + |x - A_i|$. Phần sườn dốc > 0 bị san phẳng về độ dốc 0.
- **Bậc 3 (Cài đặt Max-Heap):** Khởi tạo `priority_queue<long long> L`. Duyệt qua từng phần tử $x \in A$: nếu $x < L.top()$, cộng dồn chi phí $L.top() - x$, `L.pop()`, đẩy x hai lần. Ngược lại đẩy x một lần. Bộ nhớ chỉ $O(N)$, thời gian $O(N \log N)$.

Bài 6: [AtCoder ARC 070 E - NarrowRectangles](#)

- **Nguồn:** [AtCoder Regular Contest 070 — Task E](#).
- **Giới hạn:** $N \leq 10^5$, $1 \leq l_i < r_i \leq 10^9$, Thời gian: 2.0s, Bộ nhớ: 256 MB.
- **Tóm tắt bài toán:** Có N hình chữ nhật có độ cao 1, hình thứ i trải từ hoành độ l_i đến r_i . Bạn được phép trượt ngang các hình chữ nhật. Tìm tổng khoảng cách di chuyển nhỏ nhất sao cho hai hình chữ nhật kề nhau bất kỳ $(i, i + 1)$ luôn có phần giao nhau theo phương thẳng đứng.

Gợi ý 3 bậc:

- **Bậc 1 (Tham số hóa trạng thái):** Gọi x_i là hoành độ mép trái mới của hình chữ nhật i (chiều dài cố định $w_i = r_i - l_i$). Ràng buộc giao nhau nghĩa là: $x_i - w_{i+1} \leq x_{i+1} \leq x_i + w_i$.
- **Bậc 2 (Phép mở rộng cửa sổ - Window dilation):** Đặt hàm $dp_i(x)$ là chi phí tối thiểu cho i hình đầu tiên khi hình thứ i đặt tại mép trái x . Chuyển trạng thái: $dp_{i+1}(x) = \min_{y \in [x-w_i, x+w_{i+1}]} dp_i(y) + |x - l_{i+1}|$.
- **Bậc 3 (Lazy Tags cho 2 Heaps):** Thao tác $\min_{y \in [x-a, x+b]} f(y)$ làm giãn vùng đáy phẳng của hàm lồi: các điểm bên trái dạt sang trái thêm a , các điểm bên phải dạt sang phải thêm b . Ta duy trì 2 biến trể `shift_L += w_i` và `shift_R += w_{i+1}` kết hợp với 2 Heaps L, R . Độ phức tạp $O(N \log N)$.

Bài 7: [AtCoder ABC 217 H - Snaketoy](#)

- **Nguồn:** [AtCoder Beginner Contest 217 — Task H](#).
- **Giới hạn:** $N \leq 2 \times 10^5$, $T_i \leq 10^9$, $D_i \in \{0, 1\}$, $X_i \leq 10^9$, Thời gian: 2.0s, Bộ nhớ: 1024 MB.
- **Tóm tắt bài toán:** Một chú rắn xuất phát từ vị trí 0 tại thời điểm 0, di chuyển với vận tốc tối đa 1 đơn vị/giây. Có N đợt tấn công tại thời điểm T_i : đòn $D_i = 0$ gây sát thương $\max(0, X_i - x)$, đòn $D_i = 1$ gây sát thương $\max(0, x - X_i)$. Tìm tổng sát thương nhỏ nhất có thể nhận.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện phép chuyển trạng thái lồi):** Giữa hai thời điểm T_{i-1} và T_i , thời gian trôi qua $\Delta t = T_i - T_{i-1}$, tọa độ x có thể dịch chuyển tối đa Δt . Do đó hàm chi phí được mở rộng đều ra hai phía: $\min_{y \in [x-\Delta t, x+\Delta t]} f(y)$.
- **Bậc 2 (Cập nhật sườn dốc bất đối xứng):**
 - Khi dính đòn $D_i = 0$: hàm cộng thêm $\max(0, X_i - x) \implies$ tăng độ dốc bên trái (thêm một điểm gãy vào L).
 - Khi dính đòn $D_i = 1$: hàm cộng thêm $\max(0, x - X_i) \implies$ tăng độ dốc bên phải (thêm một điểm gãy vào R).
- **Bậc 3 (Khởi tạo và cân bằng):** Ban đầu chú rắn ở vị trí 0, tương đương hàm $f_0(0) = 0$ và $f_0(x) = \infty$ khi $x \neq 0$. Ta có thể khởi tạo L và R chứa mỗi bên N phần tử số 0 để tạo vách dốc dựng đứng. Dùng lazy tag `shift_L -= dt` và `shift_R += dt`. Độ phức tạp $O(N \log N)$.

Bài 8: [Luogu P4331 - \[BalticOI 2004\] Sequence](#)

- **Nguồn:** [BalticOI 2004 / Luogu P4331](#).
- **Giới hạn:** $N \leq 10^6, t_i \leq 2 \times 10^9$, Thời gian: 1.0s, Bộ nhớ: 128 MB.
- **Tóm tắt bài toán:** Tương tự bài CF 713C, biến đổi dãy A thành dãy B tăng ngặt với tổng $\sum |A_i - B_i|$ nhỏ nhất, nhưng yêu cầu **in ra toàn bộ dãy B tối ưu**.

Gợi ý 3 bậc:

- **Bậc 1 (Tính chất đoạn phẳng):** Sau khi đổi biến $A'_i = A_i - i$, dãy tối ưu B' sẽ bao gồm các đoạn giá trị bằng nhau liên tiếp. Trong mỗi khối đoạn, giá trị tối ưu chính là trung vị của các phần tử trong khối đó!
- **Bậc 2 (Gộp khối bằng Heap):** Dùng Leftist Tree hoặc gộp các khối đơn điệu giảm. Khi một khối có trung vị nhỏ hơn khối đứng trước, ta gộp hai khối lại và tính lại trung vị mới.
- **Bậc 3 (Truy vết ngược Slope Trick):** Một cách tiếp cận thanh lịch khác với Slope Trick: lưu lại giá trị của đỉnh Max-Heap L tại bước i , gọi là v_i . Vì $B'_i \leq B'_{i+1}$, ta có thể đi ngược từ cuối về: $B'_N = v_N$, và $B'_i = \min(v_i, B'_{i+1})$. Cuối cùng cộng ngược lại $B_i = B'_i + i$. Độ phức tạp $O(N \log N)$.

3. Cấp Độ 3: Nâng Cao & Phối Hợp Cấu Trúc Dữ Liệu (Advanced & Data Structures)

Bài 9: [AtCoder ARC 123 D - Inc, Dec - Decomposition](#)

- **Nguồn:** [AtCoder Regular Contest 123 — Task D](#).
- **Giới hạn:** $N \leq 2 \times 10^5, |A_i| \leq 10^8$, Thời gian: 2.0s, Bộ nhớ: 1024 MB.
- **Tóm tắt bài toán:** Cho dãy A . Tìm hai dãy B (không giảm: $B_1 \leq B_2 \leq \dots \leq B_N$) và C (không tăng: $C_1 \geq C_2 \geq \dots \geq C_N$) sao cho $A_i = B_i + C_i$ với mọi i , và tổng $\sum_{i=1}^N (|B_i| + |C_i|)$ đạt giá trị nhỏ nhất.

Gợi ý 3 bậc:

- **Bậc 1 (Biểu diễn qua sai phân):** Đặt $b_i = B_i - B_{i-1} \geq 0$ và $c_i = C_i - C_{i-1} \leq 0$. Vì $A_i - A_{i-1} = b_i + c_i$, để $b_i \geq 0$ và $c_i \leq 0$ có độ lớn nhỏ nhất, ta chọn tham lam các bước nhảy tương ứng với phần tăng hoặc giảm của A .
- **Bậc 2 (Đưa về hàm theo 1 biến B_1):** Toàn bộ các giá trị B_i, C_i đều được cố định dưới dạng $B_i = B_1 + \text{const}_1(i)$ và $C_i = A_i - B_i = -B_1 + \text{const}_2(i)$.
- **Bậc 3 (Tổng các hàm trị tuyệt đối):** Biểu thức mục tiêu trở thành $\sum_{i=1}^N (|B_1 + \alpha_i| + |B_1 + \beta_i|)$. Đây chính là bài toán tìm trung vị của $2N$ điểm! Chỉ cần tính các hằng số và lấy trung vị trong $O(N \log N)$.

Bài 10: [Codeforces 1251E2 - Voting \(Hard Version\)](#)

- **Nguồn:** [Educational Codeforces Round 75 — Problem E2](#).
- **Giới hạn:** $N \leq 2 \times 10^5, m_i \leq N, p_i \leq 10^9$, Thời gian: 2.0s, Bộ nhớ: 256 MB.

- **Tóm tắt bài toán:** Người thứ i sẽ bỏ phiếu nếu đã có ít nhất m_i người khác bỏ phiếu, hoặc ta phải trả p_i đồng để mua phiếu của họ. Tìm số tiền tối thiểu để tất cả mọi người đều bỏ phiếu.

Gợi ý 3 bậc:

- **Bậc 1 (Duyệt ngược từ yêu cầu cao nhất):** Nhóm các cử tri theo m_i . Xét từ $m = N - 1$ giảm dần về 0.
- **Bậc 2 (Điều kiện thiếu phiếu):** Tại mức m , giả sử đã có cnt người tự động bỏ phiếu từ các nhóm nhỏ hơn cộng với số người ta đã mua trước đó. Nếu tổng số người này $< m$, ta bắt buộc phải mua thêm $m - cnt$ người nữa.
- **Bậc 3 (Tham lam lỗi với Min-Heap):** Đẩy chi phí p_i của tất cả cử tri có $m_i = m$ vào Min-Heap. Mỗi khi thiếu phiếu, lấy ra người có p_i nhỏ nhất trong Heap. Tính đúng đắn đảm bảo dựa trên tính lỗi của đường chi phí tối ưu. Độ phức tạp $O(N \log N)$.

Bài 11: [Codeforces 802N - April Fools' Problem \(Medium\)](#)

- **Nguồn:** [Helvetic Coding Contest 2017 — Problem N](#).
- **Giới hạn:** $N \leq 2200, k \leq N, a_i, b_i \leq 10^9$, Thời gian: 4.0s, Bộ nhớ: 256 MB.
- **Tóm tắt bài toán:** Có N ngày. Cần chuẩn bị và in ra đúng k bài toán. Mỗi bài toán phải được chuẩn bị vào ngày i (chi phí a_i) và in vào ngày $j \geq i$ (chi phí b_j). Mỗi ngày chỉ được chuẩn bị tối đa 1 bài và in tối đa 1 bài. Tìm chi phí nhỏ nhất.

Gợi ý 3 bậc:

- **Bậc 1 (Mô hình luồng chi phí cực tiểu - Min Cost Flow):** Mạng luồng với cạnh dung lượng 1, chi phí a_i vào và b_j ra. Do mạng luồng không có chu trình âm, hàm chi phí tối thiểu theo k , $f(k)$, là một hàm lồi (Convex function).
- **Bậc 2 (Slope Trick / Regret Greedy):** Duyệt qua từng ngày, duy trì các ngày chuẩn bị chưa được ghép và các ngày in đã ghép. Khi gặp ngày in b_j , ta có thể ghép với ngày chuẩn bị rẻ nhất, hoặc thay thế một ngày in đắt đỏ hơn trước đó.
- **Bậc 3 (Duy trì bằng 2 Priority Queues):** Tương tự như bài Buy Low Sell High nhưng có ràng buộc đúng k lần chọn. Với $N \leq 2200$, Min Cost Flow hoặc Heap đều chạy tốt trong $O(N \log N)$.

Bài 12: [AtCoder AGC 018 C - Coins](#)

- **Nguồn:** [AtCoder Grand Contest 018 — Task C](#).
- **Giới hạn:** $N = X + Y + Z \leq 10^5, A_i, B_i, C_i \leq 10^9$, Thời gian: 2.0s, Bộ nhớ: 256 MB.
- **Tóm tắt bài toán:** Có N người. Cần chọn đúng X người nhận đồng vàng (giá trị A_i), Y người nhận đồng bạc (B_i), và Z người nhận đồng đồng (C_i). Mỗi người nhận đúng 1 loại. Tìm tổng giá trị lớn nhất.

Gợi ý 3 bậc:

- **Bậc 1 (Chuẩn hóa về 2 biến):** Trừ tất cả giá trị cho C_i : xem như ban đầu ai cũng nhận đồng C . Bài toán quy về chọn X người nhận thêm $A_i - C_i$ và Y người nhận thêm $B_i - C_i$.

- **Bậc 2 (Sắp xếp theo hiệu số):** Sắp xếp N người theo độ ưu tiên giảm dần của $(A_i - B_i)$. Trong cấu hình tối ưu, luôn tồn tại một ngưỡng chia cắt: những người nhận vàng sẽ nằm ở nửa trước, những người nhận bạc sẽ nằm ở nửa sau!
- **Bậc 3 (Hai đầu tiên tố - hậu tố bằng Heap):**
- Dùng Min-Heap tiền tố duy trì X người có $A_i - C_i$ lớn nhất trong $1 \dots i$.
- Dùng Min-Heap hậu tố duy trì Y người có $B_i - C_i$ lớn nhất trong $i + 1 \dots N$.
- Duyệt qua mọi vị trí chia cắt $i \in [X, N - Y]$ để tìm tổng cực đại. Độ phức tạp $O(N \log N)$.

4. Cấp Độ 4: Chinh Phục & Slope Trick Trên Cây (Mastery & Tree Slope Trick)

Bài 13: [Luogu P3642 / APIO 2016 - Fireworks \(烟花表演\)](#)

- **Nguồn:** [APIO 2016 / Luogu P3642](#).
- **Giới hạn:** $N + M \leq 3 \times 10^5$, $c_i \leq 10^9$, Thời gian: 1.0s, Bộ nhớ: 256 MB.
- **Tóm tắt bài toán:** Cho một cây có gốc, mỗi cạnh có trọng số c_e . Bạn có thể tăng hoặc giảm trọng số mỗi cạnh về giá trị nguyên không âm bất kỳ với chi phí bằng độ chênh lệch. Tìm chi phí tối thiểu để khoảng cách từ gốc đến tất cả các lá đều bằng nhau.

Gợi ý 3 bậc:

- **Bậc 1 (Hàm chi phí lồi tại mỗi nút):** Gọi $f_u(x)$ là chi phí nhỏ nhất để tất cả các lá trong cây con gốc u có khoảng cách đến u đúng bằng x . $f_u(x)$ là tổng hàm lồi của các con, sau đó dịch chuyển qua cạnh nối với cha.
- **Bậc 2 (Biến đổi hàm qua một cạnh):** Nếu con v nối với cha qua cạnh độ dài w , hàm $f_v(x)$ biến thành:

$$g_v(x) = \min_{0 \leq y \leq x} (f_v(y) + |x - y - w|)$$

Thao tác này tương đương với việc: cắt bỏ các điểm dốc > 1 ở sườn phải, dịch chuyển đáy phẳng sang phải w đơn vị, và kéo dài sườn dốc nghiêng $+1$ ra vô tận!

- **Bậc 3 (Hòa nhập Heap có thể trộn - Leftist Tree / Skew Heap):**
- Mỗi nút lưu danh sách các điểm gãy dốc bằng một Max-Heap có thể trộn (Meldable Heap).
- Khi gộp các con của u , ta chỉ việc `merge` các Heap lại với nhau.
- Khi rút gọn hàm qua cạnh: liên tục `pop` các điểm gãy vượt quá số con cho đến khi hệ số góc còn lại đúng $+1$, sau đó thêm 2 điểm gãy mới tương ứng với đoạn dịch chuyển w . Cuối cùng tính $f(0)$ và truy ngược về cực tiểu. Tổng thời gian $O((N + M) \log(N + M))$.

Bài 14: [Codeforces 802O - April Fools' Problem \(Hard\)](#)

- **Nguồn:** [Helvetic Coding Contest 2017 — Problem O](#).
- **Giới hạn:** $N \leq 5 \times 10^5$, $k \leq N$, $a_i, b_i \leq 10^9$, Thời gian: 10.0s, Bộ nhớ: 256 MB.

- **Tóm tắt bài toán:** Tương tự bài CF 802N nhưng với ràng buộc N lên đến 5×10^5 . Giải thuật $O(N^2)$ hay Luồng thông thường đều bị TLE.

Gợi ý 3 bậc:

- **Bậc 1 (Kết hợp Alien's Trick / WQS Binary Search):** Vì hàm chi phí tối thiểu $f(k)$ là hàm lồi theo k , ta áp dụng phương pháp nhân tử Lagrange / Alien's Trick: Chặt nhị phân chi phí phạt λ cho mỗi bài toán được ghép.
- **Bậc 2 (Bài toán con không giới hạn số lượng):** Với một giá trị λ cố định, ta cần giải bài toán tìm $\min \sum (a_i + b_j - \lambda)$ không giới hạn số lượng cặp ghép k .
- **Bậc 3 (Slope Trick / Regret Greedy trong $O(N \log N)$):** Sử dụng Priority Queue để giải bài toán con với giá phạt λ trong $O(N \log N)$. Khi số cặp ghép đạt đúng $\geq k$, ta cập nhật nghiệm. Tổng độ phức tạp: $O(N \log N \log(\max C))$.

Bài 15: [Luogu P5324 - 【BJOI2019】删除 \(Delete\)](#)

- **Nguồn:** [BJOI 2019 / Luogu P5324](#).
- **Giới hạn:** $N, M \leq 1.5 \times 10^5$, Thời gian: 2.0s, Bộ nhớ: 512 MB.
- **Tóm tắt bài toán:** Cho dãy A . Mỗi bước được xóa một phần tử hoặc sửa một phần tử. Hỗ trợ M truy vấn sửa đổi điểm và dịch chuyển toàn bộ dãy $A_i \leftarrow A_i + 1$ hoặc $A_i \leftarrow A_i - 1$. Sau mỗi truy vấn, tính số phần tử ít nhất cần xóa để các phần tử còn lại có thể sắp xếp thành dãy liên tiếp $1, 2, \dots, k$.

Gợi ý 3 bậc:

- **Bậc 1 (Mô hình phủ đoạn):** Đếm tần số xuất hiện $cnt[x]$. Mỗi giá trị x có thể phủ đoạn $[x - cnt[x] + 1, x]$. Nếu các đoạn phủ kín được khoảng $[1, k]$ không có khe hở, ta có thể giữ lại các phần tử đó.
- **Bậc 2 (Độ dốc và số lỗ hổng):** Chi phí cần xóa chính là tổng số phần tử ban đầu trừ đi số vị trí trong $[1, n]$ được phủ bởi ít nhất một đoạn. Biểu diễn hàm phủ độ bao phủ.
- **Bậc 3 (Segment Tree duy trì giá trị nhỏ nhất và số lượng):** Xây dựng Cây phân đoạn (Segment Tree) hỗ trợ cộng đoạn và đếm số lượng vị trí bằng 0. Dịch chuyển toàn bộ mảng tương đương với việc dịch chuyển gốc tọa độ của Segment Tree trong $O(1)$. Tổng độ phức tạp $O((N + M) \log N)$.

Bài 16: [Codeforces 1148E - Earth Wind and Fire](#)

- **Nguồn:** [Codeforces Round 566 \(Div. 2\) — Problem E](#).
- **Giới hạn:** $N \leq 3 \times 10^5$, $s_i, t_i \leq 10^9$, Thời gian: 4.0s, Bộ nhớ: 256 MB.
- **Tóm tắt bài toán:** Cho tọa độ ban đầu S và tọa độ đích T của N hòn đá trên một đường thẳng. Trong mỗi thao tác, chọn 2 hòn đá i, j có $s_i \leq s_j$ và một số $d > 0$ sao cho $s_i + d \leq s_j - d$, rồi di chuyển hòn đá i sang phải d bước và hòn đá j sang trái d bước. Hãy tìm chuỗi thao tác để đưa S về T hoặc báo không thể.

Gợi ý 3 bậc:

- **Bậc 1 (Bất biến thứ tự & Sắp xếp):** Do hai hòn đá di chuyển lại gần nhau và không được vượt qua nhau, thứ tự tương đối của các hòn đá không bao giờ thay đổi. Sắp xếp cả S và T theo thứ tự tăng dần.
- **Bậc 2 (Bổ đề tổng tích lũy - Prefix Sums):** Đặt $D_i = t_i - s_i$. Điều kiện cần và đủ để tồn tại lời giải: $\sum_{i=1}^N D_i = 0$ (tổng khoảng cách không đổi).
- Với mọi tiền tố $k \in [1, N]$: $\sum_{i=1}^k s_i \leq \sum_{i=1}^k t_i \iff \sum_{i=1}^k D_i \geq 0$.
- **Bậc 3 (Ghép cặp bằng Stack hai con trỏ):** Duyệt từ trái sang phải. Dùng một Stack lưu các vị trí i cần di chuyển sang phải ($D_i > 0$). Khi gặp vị trí j cần di chuyển sang trái ($D_j < 0$), ta ghép cặp đỉnh Stack với j để thực hiện thao tác triệt tiêu khoảng cách cho đến khi $D_j = 0$. Số thao tác không vượt quá $5N$. Độ phức tạp $O(N \log N)$.



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.